

Лабораторна робота №1.

Локальний сервінг і надійний typed-клієнт

26 серпня 2026 р.

Мета

Розгорнути дві локальні мовні моделі, виміряти їхню поведінку під навантаженням і побудувати навколо них сервіс, який перетворює недовірений текст на валідований типізований об'єкт або на контрольовану відмову — і жодного разу не повертає неперевірений результат як успішний.

Робота виконується **бригадою з трьох осіб**. Лекції 1–5 і 9.

Усе безкоштовне. Жодного платного API: моделі запускаються локально через Ollama або llama.cpp, усі бібліотеки відкриті. Якщо в когось із бригади немає відеокарти, працює CPU-профіль: модель 0,5–1,5B і контекст 2k.

Склад бригади і розподіл ролей

Кожен учасник відповідає за свою підсистему, має власний результат і захищає його окремо. Спільним є лише інтеграційний крок і підсумковий звіт.

- **Учасник А — сервінг і вимірювання.** Розгортає дві моделі, знімає TTFT, TROT і пропускну здатність, рахує бюджет пам'яті за формулами і звіряє з фактом.
- **Учасник В — контракт і розбір.** Описує схему результату, вмикає обмежене декодування, реалізує розбір із однією спробою відновлення і класифікацію помилок за чотирма класами.
- **Учасник С — надійність і перевірка.** Реалізує політику повторів, таймауту, бюджет часу та ідемпотентність; складає набір випадків і пише тести.

Інтеграційний крок виконують разом: клієнт учасника В і механізми учасника С загортаються у HTTP-сервіс поверх сервінгу учасника А, після чого набір учасника С проганяється на обох моделях учасника А.

Теоретичні відомості

Мовна модель повертає розподіл над продовженнями, а не значення функції. Та сама пара «інструкція + запит» може дати різні виходи, і причина не лише в семплінгу: на результат впливають версія моделі, склад і порядок контексту, а на GPU ще й пакетування запитів разом із неасоціативністю операцій з рухомою комою. Тому текст відповіді не є інтерфейсом, а межа між моделлю і кодом описується контрактом.

- **Дві фази запиту мають різну природу.** Префіл обробляє весь запит одразу і впирається в обчислення; декод породжує токени по одному і впирається в пропускну

здатність пам'яті, бо на кожен токен перерахуються всі ваги. Звідси дві різні метрики: час до першого токена (TTFT) залежить від довжини запиту, час на наступний токен (TPOT) — від пам'яті й розміру моделі. Наскрізна затримка складається як $T = TTFT + TPOT \cdot N_{out}$.

- **Пам'ять ділиться на ваги і KV-кеш:**

$$S_{KV} = 2 \cdot L \cdot n_{kv} \cdot d_h \cdot b \cdot s \cdot B$$

де L — шарів, n_{kv} — KV-голів, d_h — вимір голови, b — байтів на число, s — довжина, B — батч. Саме залишок пам'яті під кеш, а не заявлена довжина контексту, задає кількість одночасних сеансів.

- **Гарантій формату є три рівні, і вони не взаємозамінні.** Інструкція в промпті «відповідай у JSON» не гарантує нічого. Валідація після генерації гарантує, що некоректний результат не пройде далі, ціною повторних викликів. Обмежене декодування буде ґраматику допустимих продовжень і забороняє токени, що виводять за межі схеми; синтаксична валідність тоді гарантована процедурою генерації. Локально цю роль виконують GBNF-ґраматика в `llama.cpp` або параметр `format` в Ollama.
- **Схема описує форму, а не зміст.** Заборона зайвих полів (`additionalProperties: false`) не косметична: без неї модель домальовує поля, яких немає в контракті, і вони мовчки проходять далі. Але навіть строга схема гарантує лише те, що поле існує і має потрібний тип. Доменні правила перевіряє код.
- **Чотири класи помилок вимагають чотирьох різних реакцій.** *Транспортна* каже, що виклик не дійшов, — її повторюють із затримкою. *Схемна* означає, що відповідь не лягла в контракт, — повторюють не більше двох разів, передаючи моделі текст помилки валідатора. *Доменна* означає, що відповідь валідна за формою і неприйнятна за правилами задачі, — повтор не допоможе. *Відмова моделі* є законною відповіддю, а не збоєм.
- **Ідемпотентність робить повтор безпечним.** Ключ операції обчислюється до виклику з тих даних, що визначають операцію, і зберігається разом із результатом. Без ключа будь-який ретрай на дії зі станом подвоює наслідок.

Корисні посилання для ознайомлення:

- [Каталог моделей Ollama і FAQ зі змінними оточення](#);
- [Pydantic: JSON Schema та Strict Mode](#);
- llama.cpp: [GBNF Grammars](#);
- M. Brooker, [Exponential Backoff And Jitter](#);
- [FastAPI](#) для сервісної обгортки та [Locust](#) для навантажувального тесту.

Порядок виконання

Позначка `[A]`, `[B]`, `[C]` вказує відповідального за крок; `[разом]` — крок виконує вся бригада.

1. **[разом] Узгодити контракт результату** — предметну область і структуру відповіді з не менш ніж п'ятьма полями. Контракт фіксується до написання коду і далі не змінюється без спільного рішення.

2. **[A] Розгорнути дві локальні моделі** різного розміру і переконатися, що обидві працюють у передбачуваний конфігурації, а не на значеннях за замовчуванням.
3. **[A] Виміряти затримки і пропускну здатність** обох моделей за кількох рівнів конкурентності та порівняти розрахований бюджет пам'яті з фактичним споживанням. Результат — таблиця p50 і p95 для обох моделей.
4. **[B] Реалізувати контракт кодом** і домогтися, щоб модель фізично не могла повернути іншу форму. Показати, скільки невалідних відповідей дає та сама модель без цього механізму і з ним.
5. **[B] Реалізувати розбір відповіді** з відновленням після помилки формату і класифікацією збоїв за чотирма класами з лекції. Для кожного класу — типізована відмова з рішенням про повтор.
6. **[C] Зробити клієнт стійким до збоїв** і безпечним до повторів. Показати в логах, що обробку зупиняє власна політика клієнта, а не виняток, і що повторений запит не створює другого запису.
7. **[C] Скласти набір не менш ніж із 40 випадків** і покрити його тестами. Усі тести проходять без мережі та без ключа API.
8. **[разом] Зібрати сервіс** поверх обох моделей, прогнати на ньому набір і навантажити його так, щоб спрацювали таймаут, залобіжник і резервна модель.
9. **[разом] Оформити ноутбук**. Зібрати роботу в один Jupyter-ноутбук: код кожного учасника у своїх комітках, поруч — короткі коментарі, що саме робить блок і який результат очікується. Обов'язково всередині: JSON Schema контракту, порівняльна таблиця двох моделей, таблиця чотирьох класів помилок із частотою кожного, результат навантажувального тесту і висновок до 250 слів про те, яку модель ви обрали б для навчального відділу і чому.

Контрольні запитання

1. Чому температура 0 не гарантує однакового виходу на однаковому вході?
2. Чому декод упирається в пропускну здатність пам'яті, а не в обчислення?
3. Що саме гарантує обмежене декодування і чого воно не гарантує?
4. Навіщо в схемі `additionalProperties: false`, якщо зайві поля можна ігнорувати?
5. Чим доменна помилка відрізняється від помилки схеми і чому реакція на них різна?
6. У яких випадках повторний виклик робить гірше, ніж відмова?
7. Що станеться, якщо у політиці повтору немає випадкової складової?
8. Як обчислити ключ ідемпотентності для запиту з довільним текстом користувача?
9. Чому p95 зростає нелінійно за високого завантаження?
10. Ваша менша модель дала вищу якість на наборі, ніж більша. Які пояснення можливі й як їх перевірити?
11. Чому набір випадків краще скласти одразу після контракту, а не після написання коду?

Література

1. Willard B., Louf R. [Efficient Guided Generation for Large Language Models](#). arXiv:2307.09702, 2023.
2. Kwon W. та ін. [Efficient Memory Management for LLM Serving with PagedAttention](#). SOSP, 2023.
3. Xiao T., Zhu J. [Foundations of Large Language Models](#). arXiv:2501.09223, 2025. Розділ 5.
4. Jurafsky D., Martin J. H. [Speech and Language Processing](#), 3rd ed. draft, 2026. Розділ 7.